Lecture 9 — Dependency injection and Spring bean wiring

Lecture 9 covered dependency injection and bean wiring. H2 storage, request validation and shared error handling continue in Lecture 10.

Hospital API: from working endpoints to connected objects, clear errors and repeatable tests.

0. Movie Watchlist — homework problem revisited

Build a small REST API for a movie watchlist. Store movies in memory. Each movie contains a server-generated id, a title and a releaseYear. Implement your own model and controller using the CRUD patterns from the Hospital application.

Required endpoints

```
GET    /movies       -> 200 and a JSON array, including []
GET    /movies/{id}  -> 200 and the movie, or 404
POST   /movies       -> 201, Location header and created movie
PUT    /movies/{id}  -> 200 and replacement movie, or 404
DELETE /movies/{id}  -> 204 with no body, or 404
```

Create/update request

```
{
  "title": "Interstellar",
  "releaseYear": 2014
}
```

The client sends title and releaseYear only. The server generates the ID. PUT replaces both details of an existing movie and preserves its ID; it must not create a movie for a missing ID.

Rules

• Title is required and must not be blank.

• releaseYear is required and must be between 1888 and 2100 inclusive.

• Invalid input returns 400 with a useful message. Validate before writing data or allocating an ID.

• An invalid update must leave the existing movie unchanged.

• Duplicate titles are allowed; movie identity is its ID.

• Do not generate IDs using collection size + 1. Deleting a movie must not cause a later create to overwrite another movie.

• Data may reset when the application restarts. No database, UI, authentication, service/repository layers or dependency injection is required for this assignment.

Check your implementation

• Create two movies and confirm their IDs are distinct. Read the collection and each movie.

• Update one movie: its ID stays the same and both details change.

• Send an invalid update, then GET the movie and confirm its old details remain.

• Delete an early ID, create another movie and confirm no remaining movie was overwritten.

• Read/delete a missing or already-deleted ID: 404.

• Delete all movies: GET /movies returns 200 with [].

• Repeat the same POST and the same PUT; compare their effects on stored state.

Discuss before viewing the solution

• Where should validation happen relative to ID allocation and storage changes?

• After deleting ID 1 while ID 2 remains, what goes wrong with size + 1?

• Why does an update keep the existing ID?

• How would you locate a movie by ID using an ArrayList? How would a map change that lookup? Either collection can satisfy this assignment.

• Why does POST return 201, while successful DELETE returns 204 without a body?

• What happens to this in-memory watchlist when the application restarts?

Submission: source code, brief example requests and Postman evidence or screenshots covering the acceptance sequence. Search, sorting and pagination are optional extras after the core CRUD works.

1. Controller, service and repository

Controller receives HTTP input and returns HTTP responses. Service performs application

operations and business checks. Repository reads/writes stored patients. These responsibilities are a design choice; annotations alone do not enforce good separation.

```
HTTP request -> HospitalController -> HospitalPatientService -> Hospit
alPatientRepository
```

2. Dependency injection in ordinary Java

A dependency is another object a class needs. Dependency injection supplies that object from outside. It also works without Spring: constructing a repository and passing it to the service is manual constructor injection.

```
PatientRepository repository = new PatientRepository();
PatientService service = new PatientService(repository);
System.out.println(service.getPatientName(1));
repository.rename(1, "Meera");
System.out.println(service.getPatientName(1));
```

The service receives the same repository reference, not a copied database. A separate new PatientRepository() has separate storage. final on a reference prevents reassignment; it does not make the referenced map/list immutable.

3. Spring beans, IoC and automatic injection

A bean is an object managed by a Spring container. IoC delegates object assembly to the container. DI is the supply of required collaborators. Registering an object, retrieving it and injecting it are related but different operations.

@Component marks a general component; @Service communicates service responsibility; @Repository communicates data-access responsibility. @Controller identifies MVC controllers; @RestController combines controller and response-body behavior. URL mappings are registered by Spring at runtime, not by the Java compiler.

```
@Service
public class PatientService {
    private final PatientRepository repository;

    public PatientService(PatientRepository repository) {
        this.repository = repository;
    }
}
```

For one constructor, @Autowired is optional. Field/setter injection are alternatives; use one approach at a time. With constructor injection, required dependencies are explicit. A missing required bean can prevent startup.

```
try (var context = new AnnotationConfigApplicationContext(
        "com.accio.springProject.DiInjection")) {
    PatientService service = context.getBean(PatientService.class);
    PatientRepository repository = context.getBean(PatientRepository.c
lass);
    System.out.println(service.getPatientName(1));
    repository.rename(1, "Meera");
    System.out.println(service.getPatientName(1));
}
```

This console example explicitly starts a context and retrieves beans. Spring injects repository into service when constructing the service; it does not populate local variables in static main

automatically. In the web app, SpringApplication.run starts the Boot context and embedded server. Default scanning covers the main application package and its subpackages. new Patient(...) is not automatically bean registration.

4. Refactor our ArrayList Hospital application

The active classes are under com.accio.springProject.hospitalManagement: Controller/HospitalController, Service/HospitalPatientService, Repository/HospitalPatientRepository, Model/Patient and Entity/PatientInput. PatientInput is a request DTO despite the existing Entity folder name; JDBC does not require JPA @Entity. MainController is old commented reference code.

Move the same ArrayList, Patient1/25 and Patient2/32 seeds, ID counter and mutations into HospitalPatientRepository. Inject it into HospitalPatientService, then inject that service into HospitalController. Storage starts as an ArrayList; DI does not require switching to a map.

The point-4 repository methods are findAll(), add(PatientInput), updateName(long, PatientInput) and delete(long). Search and sort belong to the service; request annotations and ResponseEntity belong to the controller. Remove duplicate controller storage once the repository owns it.

```
GET    /hospital/patients
POST   /hospital/addPatient
PUT    /hospital/updatePatient/{id}
DELETE /hospital/deletePatients/{id}
GET    /hospital/search?name=Patient1
GET    /hospital/browse?param=name
```

Successful writes return the full patient list with 200. Update changes name only; age and ID stay unchanged. Search is exact and case-sensitive. Browse sorts by id/name, otherwise age. Equal names remain in the list: sorting does not remove duplicates. Safe deletion uses an index and returns immediately after removal. The counter is independent of list size.

5. Explicit bean registration: @Configuration and @Bean

Problem: generate a patient greeting. MessageFormatter defines String format(String name). ShortFormatter implements it and returns "Hi " plus the name. Register its object explicitly and print the result.

```
// MessageFormatter.java
public interface MessageFormatter {
    String format(String name);
}

// ShortFormatter.java
public class ShortFormatter implements MessageFormatter {
    public String format(String name) {
        return "Hi " + name;
    }
}

// BeanLabConfiguration.java
@Configuration
public class BeanLabConfiguration {
    @Bean
    public MessageFormatter shortFormatter() {
```

```
        return new ShortFormatter();
    }
}
```
Each public class belongs in a separate file in the same springBeans package. Import org.springframework.context.annotation.Bean and Configuration. @Configuration identifies a class containing configuration/bean definitions. @Bean marks a factory method; Spring manages its returned object. The method name is the default bean name. @Bean is not a class annotation.
```
try (var context = new AnnotationConfigApplicationContext(BeanLabConfi
guration.class)) {
    MessageFormatter formatter = context.getBean(MessageFormatter.clas
s);
    System.out.println(formatter.format("Asha")); // Hi Asha
}
```
This explicitly retrieves a registered bean; it is not automatic injection into a main-method variable. Multiple @Bean methods or configuration classes are allowed. @Bean is useful for library objects and custom construction; @Component is convenient for annotating your own class. Avoid registering ShortFormatter through BOTH @Component and @Bean in the same context. Package scanning also discovers eligible components and configuration classes.

6. Multiple implementations: @Primary and @Qualifier

DetailedFormatter also implements MessageFormatter and returns "Welcome to the clinic, " plus the name. Two matching beans can make an unqualified lookup/injection ambiguous. @Primary establishes a default; @Qualifier requests a particular candidate for one dependency. A qualifier does not change dynamically for each HTTP request.
```
@Bean
@Primary
public MessageFormatter shortFormatter() {
    return new ShortFormatter();
}

@Bean
public MessageFormatter detailedFormatter() {
    return new DetailedFormatter();
}

@Bean
public ReportWriter reportWriter(
        @Qualifier("detailedFormatter") MessageFormatter formatter) {
    return new ReportWriter(formatter);
}
public class ReportWriter {
    private final MessageFormatter formatter;
    public ReportWriter(MessageFormatter formatter) {
        this.formatter = formatter;
    }
    public String write(String name) {
        return formatter.format(name);
    }
}
```

A type-only MessageFormatter lookup gets the primary short formatter. ReportWriter receives the detailed formatter because its factory argument is qualified. Depend on the interface so either implementation can satisfy the same formatting capability.

Continuation in Lecture 10

1. H2 SQL storage (formerly Lecture 9 point 7).

2. Request validation (formerly point 8).

3. Domain failures and shared error responses (formerly point 9).